

10. Intrusion Detection System Basics: Analyze intrusion detection techniques and simulate detection of malicious activities

Aim

To understand the basic concepts and techniques of Intrusion Detection Systems (IDS) and implement a Python-based IDS simulation in Google Colab for detecting suspicious network activities.

Objectives

After completing this practical, students will be able to:

1. Understand the purpose and role of an Intrusion Detection System.
2. Differentiate between **Network-based IDS (NIDS)** and **Host-based IDS (HIDS)**.
3. Understand **signature-based** and **anomaly-based** intrusion detection techniques.
4. Analyze network traffic characteristics for identifying suspicious behavior.
5. Define normal and suspicious traffic patterns using thresholds.
6. Simulate network traffic containing normal and malicious activities.
7. Detect possible **DoS-like traffic, port scanning, and suspicious IP activity**.
8. Generate intrusion alerts based on predefined detection rules.
9. Maintain an intrusion detection log using Python.
10. Visualize detected activities using graphs.
11. Understand the importance of false positives and false negatives in IDS.
12. Implement a basic IDS simulation using Python in **Google Colab**.

1. Introduction to Intrusion Detection System

An **Intrusion Detection System (IDS)** is a security mechanism used to monitor network or system activities and identify events that may indicate unauthorized access, attacks, misuse, or other malicious behavior.

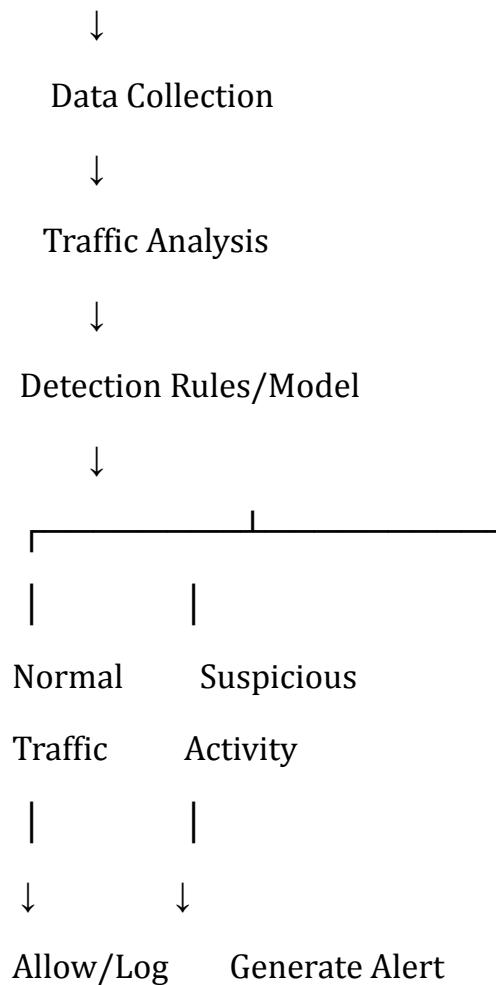
An IDS continuously analyzes available activity data such as:

- Network traffic
- Source and destination IP addresses
- Ports
- Number of requests
- Connection attempts
- Login attempts
- Protocol information
- System events

When suspicious activity is identified, the IDS generates an **alert** for further investigation.

Basic IDS Process

Network/System Activity



2. Types of Intrusion Detection Systems

A. Network-Based IDS (NIDS)

A **Network-Based Intrusion Detection System** monitors network traffic to identify suspicious or malicious activities.

Examples:

- Excessive requests
- Port scanning
- Suspicious connections
- Unusual traffic volume
- Known attack patterns

Example:

An IDS observes that one IP address is making hundreds of connection attempts to different ports of a server.

B. Host-Based IDS (HIDS)

A **Host-Based Intrusion Detection System** monitors activities occurring on an individual computer or server.

It may analyze:

- Login attempts
- File modifications
- System logs
- Application logs
- Process activity
- Configuration changes

Example:

A server log shows multiple failed login attempts followed by a successful login from an unusual source.

3. Intrusion Detection Techniques

3.1 Signature-Based Detection

Signature-based IDS compares observed activity with previously known attack patterns or signatures.

Example

If a known malicious pattern is:

More than 100 requests from the same source

then traffic exceeding this pattern can generate an alert.

Advantages

- Effective for known attacks
- Relatively simple to implement
- Lower computational complexity

Limitation

It may not effectively detect previously unknown attacks.

3.2 Anomaly-Based Detection

Anomaly-based IDS establishes a baseline for normal behavior and identifies activities that significantly deviate from that baseline.

For example:

Normal requests = 5–20

Suspicious requests = >50

If an IP suddenly generates 150 requests, the IDS can classify the behavior as anomalous.

Advantages

- Can detect unusual or previously unknown behavior
- Useful for behavioral monitoring

Limitation

- May generate false positives
- Requires appropriate thresholds or training data

4. Basic Detection Rules for This Practical

For this Google Colab simulation, the IDS will use simple rule-based detection.

Rule 1: Excessive Traffic

IF request_count > 50

THEN possible DoS-like activity

Rule 2: Port Scanning

IF unique_ports_scanned >= 5

THEN possible port scanning

Rule 3: Untrusted Source

IF source_IP is not in trusted_IP_list

THEN suspicious source

Rule 4: Combined Suspicious Activity

If multiple indicators are detected, the IDS will assign a higher severity.

No alert → NORMAL

One indicator → MEDIUM

Multiple → HIGH

Safety note: This practical only generates and analyzes **synthetic traffic records**. It does not perform an actual DoS attack, port scan, packet injection, or unauthorized network activity.

5. Google Colab Practical

```
# =====  
# PRACTICAL 10  
# INTRUSION DETECTION SYSTEM (IDS) BASICS
```

```

#
# Aim:
# Analyze intrusion detection techniques and simulate detection
# of malicious activities in a controlled Google Colab environment.
#
# Activities Simulated:
# 1. Normal Network Traffic
# 2. DoS-like Excessive Traffic
# 3. Port Scanning-like Activity
# 4. Untrusted Source Activity
#
# NOTE:
# This is ONLY a controlled simulation using synthetic data.
# No real network attack, packet injection, port scanning,
# or unauthorized activity is performed.
# =====

# =====
# 1. IMPORT REQUIRED LIBRARIES
# =====

import random
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

print("=" * 70)
print("INTRUSION DETECTION SYSTEM - GOOGLE COLAB SIMULATION")
print("=" * 70)

print("\nLibraries imported successfully.")

# =====
# 2. IDS CONFIGURATION
# =====

# Trusted IP addresses
trusted_ips = [
    "10.0.0.10",
    "10.0.0.11",
    "10.0.0.12",
    "10.0.0.13"
]

```

```

# Server being monitored
server_ip = "10.0.0.50"

# Detection thresholds
DOS_THRESHOLD = 50
PORT_SCAN_THRESHOLD = 5

print("\nIDS CONFIGURATION")
print("-" * 70)

print("Monitored Server :", server_ip)
print("Trusted IPs      :", ", ".join(trusted_ips))
print("DoS Threshold    :", DOS_THRESHOLD)
print("Port Scan Limit   :", PORT_SCAN_THRESHOLD)

# =====
# 3. GENERATE SIMULATED NETWORK TRAFFIC
# =====

records = []

start_time = datetime.now()

# -----
# A. NORMAL TRAFFIC
# -----

for i in range(30):

    source_ip = random.choice(trusted_ips)

    request_count = random.randint(5, 20)

    unique_ports = random.randint(1, 3)

    destination_port = random.choice([22, 80, 443])

    records.append({
        "timestamp": start_time + timedelta(seconds=i),
        "source_ip": source_ip,
        "destination_ip": server_ip,
        "destination_port": destination_port,
        "request_count": request_count,
        "unique_ports": unique_ports,
    })

```

```

        "activity_type": "Normal Traffic"
    })

# -----
# B. SIMULATED DOS-LIKE TRAFFIC
# -----

for i in range(8):

    source_ip = "10.0.0.99"

    request_count = random.randint(80, 150)

    unique_ports = random.randint(1, 2)

    destination_port = 80

    records.append({
        "timestamp": start_time + timedelta(seconds=40 + i),
        "source_ip": source_ip,
        "destination_ip": server_ip,
        "destination_port": destination_port,
        "request_count": request_count,
        "unique_ports": unique_ports,
        "activity_type": "DoS-like Activity"
    })

# -----
# C. SIMULATED PORT SCANNING
# -----

for i in range(7):

    source_ip = "10.0.0.77"

    request_count = random.randint(5, 20)

    unique_ports = random.randint(6, 15)

    destination_port = random.choice(
        [21, 22, 23, 25, 53, 80, 110, 135, 139, 443]
    )

    records.append({

```

```

        "timestamp": start_time + timedelta(seconds=50 + i),
        "source_ip": source_ip,
        "destination_ip": server_ip,
        "destination_port": destination_port,
        "request_count": request_count,
        "unique_ports": unique_ports,
        "activity_type": "Port Scanning-like Activity"
    })

# -----
# D. SIMULATED UNTRUSTED SOURCE ACTIVITY
# -----

untrusted_ips = [
    "192.168.1.200",
    "172.16.0.25",
    "192.0.2.55",
    "198.51.100.25"
]

for i in range(8):

    source_ip = random.choice(untrusted_ips)

    request_count = random.randint(10, 30)

    unique_ports = random.randint(1, 4)

    destination_port = random.choice([22, 80, 443])

    records.append({
        "timestamp": start_time + timedelta(seconds=60 + i),
        "source_ip": source_ip,
        "destination_ip": server_ip,
        "destination_port": destination_port,
        "request_count": request_count,
        "unique_ports": unique_ports,
        "activity_type": "Untrusted Source Activity"
    })

# =====
# 4. CREATE DATAFRAME
# =====

```



```

random.shuffle(records)

df = pd.DataFrame(records)

print("\n" + "=" * 70)
print("SIMULATED NETWORK TRAFFIC GENERATED")
print("=" * 70)

print("\nTotal Traffic Records:", len(df))

display(df.head(10))

# =====
# 5. DEFINE IDS DETECTION FUNCTION
# =====

def detect_intrusion(row):

    alerts = []

    # -----
    # RULE 1: DoS-LIKE TRAFFIC DETECTION
    # -----

    if row["request_count"] > DOS_THRESHOLD:
        alerts.append("DoS-like Traffic")

    # -----
    # RULE 2: PORT SCANNING DETECTION
    # -----

    if row["unique_ports"] >= PORT_SCAN_THRESHOLD:
        alerts.append("Port Scanning")

    # -----
    # RULE 3: UNTRUSTED SOURCE DETECTION
    # -----

    if row["source_ip"] not in trusted_ips:
        alerts.append("Untrusted Source")

    # -----

```

```

# DETERMINE SEVERITY
# -----

if len(alerts) == 0:

    severity = "NORMAL"
    action = "ALLOW"

elif len(alerts) == 1:

    severity = "MEDIUM"
    action = "ALERT"

else:

    severity = "HIGH"
    action = "ALERT + INVESTIGATE"

# Return detection result
if len(alerts) == 0:
    detection = "No Suspicious Activity"
else:
    detection = ", ".join(alerts)

return detection, severity, action


# =====
# 6. APPLY IDS DETECTION
# =====

results = df.apply(detect_intrusion, axis=1)

df["detected_activity"] = [result[0] for result in results]

df["severity"] = [result[1] for result in results]

df["recommended_action"] = [result[2] for result in results]


# =====
# 7. DISPLAY IDS ANALYSIS RESULTS
# =====

print("\n" + "=" * 70)

```

```

print("IDS TRAFFIC ANALYSIS")
print("=" * 70)

display(df)

# =====
# 8. GENERATE IDS ALERTS
# =====

alerts = df[df["severity"] != "NORMAL"].copy()

print("\n" + "=" * 70)
print("INTRUSION DETECTION ALERTS")
print("=" * 70)

if len(alerts) == 0:

    print("\nNo suspicious activity detected.")

else:

    for index, row in alerts.iterrows():

        print(
            f"\n[{row['severity']}] "
            f"Source IP: {row['source_ip']} | "
            f"Requests: {row['request_count']} | "
            f"Unique Ports: {row['unique_ports']} | "
            f"Detection: {row['detected_activity']} | "
            f"Action: {row['recommended_action']}"
        )

# =====
# 9. IDS SUMMARY
# =====

normal_records = df[df["severity"] == "NORMAL"]

medium_alerts = df[df["severity"] == "MEDIUM"]

high_alerts = df[df["severity"] == "HIGH"]

```

```

print("\n" + "=" * 70)
print("IDS DETECTION SUMMARY")
print("=" * 70)

print("\nTotal Records      :", len(df))
print("Normal Records       :", len(normal_records))
print("Medium Alerts         :", len(medium_alerts))
print("High Alerts           :", len(high_alerts))
print("Total Alerts          :", len(alerts))

# =====
# 10. ACTIVITY TYPE ANALYSIS
# =====

print("\n" + "=" * 70)
print("ACTIVITY TYPE ANALYSIS")
print("=" * 70)

activity_summary = df["activity_type"].value_counts()

display(
    activity_summary.reset_index(
        name="Number of Records"
    ).rename(
        columns={"index": "Activity Type"}
    )
)

# =====
# 11. SOURCE IP ANALYSIS
# =====

print("\n" + "=" * 70)
print("SOURCE IP ANALYSIS")
print("=" * 70)

source_analysis = df.groupby("source_ip").agg(

    total_requests=("request_count", "sum"),

    activity_count=("source_ip", "count"),

    maximum_requests=("request_count", "max"),

```

```

        maximum_ports=("unique_ports", "max")

).reset_index()

source_analysis["status"] = source_analysis["source_ip"].apply(

    lambda ip:
        "Trusted"
        if ip in trusted_ips
        else "Untrusted"
)

display(source_analysis)

# =====
# 12. FIND MOST SUSPICIOUS SOURCE IP
# =====

print("\n" + "=" * 70)
print("MOST SUSPICIOUS SOURCE ANALYSIS")
print("=" * 70)

if len(alerts) > 0:

    suspicious_ip_count = (
        alerts["source_ip"]
        .value_counts()
    )

    print("\nSuspicious Activity by Source IP:\n")

    print(suspicious_ip_count)

    most_suspicious_ip = suspicious_ip_count.index[0]

    print(
        "\nMost frequently detected suspicious IP:",
        most_suspicious_ip
    )

else:

```

```

        print("\nNo suspicious source IP detected.")

# =====
# 13. CREATE IDS LOG
# =====

ids_log = df[
    df["severity"] != "NORMAL"
][
    [
        "timestamp",
        "source_ip",
        "destination_ip",
        "destination_port",
        "request_count",
        "unique_ports",
        "detected_activity",
        "severity",
        "recommended_action"
    ]
].copy()

print("\n" + "=" * 70)
print("IDS SECURITY LOG")
print("=" * 70)

if len(ids_log) > 0:

    display(ids_log)

else:

    print("\nNo alerts recorded in IDS log.")

# =====
# 14. NORMAL VS SUSPICIOUS TRAFFIC
# =====

df["traffic_status"] = df["severity"].apply(

    lambda x:
        "Normal"
        if x == "NORMAL"

```

```

        else "Suspicious"
    )

traffic_status_counts = (
    df["traffic_status"]
    .value_counts()
)

plt.figure(figsize=(8, 5))

traffic_status_counts.plot(
    kind="bar"
)

plt.title("Normal vs Suspicious Network Traffic")

plt.xlabel("Traffic Status")

plt.ylabel("Number of Records")

plt.xticks(rotation=0)

plt.tight_layout()

plt.show()

# =====
# 15. ACTIVITY DISTRIBUTION GRAPH
# =====

activity_counts = df["activity_type"].value_counts()

plt.figure(figsize=(10, 5))

activity_counts.plot(
    kind="bar"
)

plt.title("Simulated Network Activity Distribution")

plt.xlabel("Activity Type")

```

```
plt.ylabel("Number of Records")
```

```
plt.xticks(rotation=25, ha="right")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 16. IDS SEVERITY GRAPH  
# =====
```

```
severity_counts = df["severity"].value_counts()
```

```
plt.figure(figsize=(8, 5))
```

```
severity_counts.plot(  
    kind="bar"  
)
```

```
plt.title("IDS Alert Severity Distribution")
```

```
plt.xlabel("Severity Level")
```

```
plt.ylabel("Number of Records")
```

```
plt.xticks(rotation=0)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 17. REQUEST COUNT ANALYSIS  
# =====
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(  
    range(len(df)),  
    df["request_count"],  
    marker="o"  
)
```



```

plt.axhline(
    y=DOS_THRESHOLD,
    linestyle="--",
    label="DoS Threshold"
)

plt.title("Network Request Count and IDS Threshold")

plt.xlabel("Traffic Record")

plt.ylabel("Request Count")

plt.legend()

plt.tight_layout()

plt.show()

# =====
# 18. PORT SCANNING ANALYSIS
# =====

plt.figure(figsize=(10, 5))

plt.plot(
    range(len(df)),
    df["unique_ports"],
    marker="o"
)

plt.axhline(
    y=PORT_SCAN_THRESHOLD,
    linestyle="--",
    label="Port Scan Threshold"
)

plt.title("Unique Ports Observed During Network Activity")

plt.xlabel("Traffic Record")

plt.ylabel("Number of Unique Ports")

plt.legend()

```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# 19. DETECTION TECHNIQUE SUMMARY  
# =====
```

```
print("\n" + "=" * 70)  
print("INTRUSION DETECTION TECHNIQUE USED")  
print("=" * 70)
```

```
print("""  
1. SIGNATURE/RULE-BASED DETECTION  
-----  
The IDS checks predefined conditions.
```

```
Rule 1:  
Request Count > 50  
-> DoS-like activity
```

```
Rule 2:  
Unique Ports >= 5  
-> Port scanning-like activity
```

```
Rule 3:  
Source IP not in trusted list  
-> Untrusted source
```

```
2. ANOMALY-BASED CONCEPT  
-----  
Normal traffic is assumed to have:
```

```
Request Count : 5 - 20  
Unique Ports  : 1 - 3
```

```
Significant deviation from these values  
is treated as suspicious behavior.
```

```
3. ALERT SEVERITY  
-----  
No suspicious indicator  
-> NORMAL
```

```

    One suspicious indicator
    -> MEDIUM

    Multiple suspicious indicators
    -> HIGH
    """)

# =====
# 20. FINAL SECURITY REPORT
# =====

print("\n" + "=" * 70)
print("FINAL IDS SECURITY REPORT")
print("=" * 70)

print("\nTotal Network Records :", len(df))

print(
    "Normal Traffic      :",
    len(normal_records)
)

print(
    "Suspicious Traffic   :",
    len(alerts)
)

print(
    "Medium Severity      :",
    len(medium_alerts)
)

print(
    "High Severity        :",
    len(high_alerts)
)

if len(alerts) > 0:

    print(
        "\nIDS Status          : SUSPICIOUS ACTIVITY DETECTED"
    )

```

```

else:

    print(
        "\nIDS Status          : NO SUSPICIOUS ACTIVITY"
    )

print("\nRecommended Security Actions:")

if len(alerts) > 0:

    print("""
1. Investigate suspicious source IP addresses.
2. Review high-severity IDS alerts.
3. Examine excessive request patterns.
4. Examine unusual port activity.
5. Review IDS logs for repeated suspicious behavior.
6. Apply firewall or access-control policies when appropriate.
7. Continue monitoring the affected system.
""")

else:

    print("""
1. Continue monitoring network traffic.
2. Maintain IDS logs.
3. Periodically review detection thresholds.
4. Update detection rules when new threats are identified.
""")

print("=" * 70)
print("PRACTICAL 10 COMPLETED SUCCESSFULLY")
print("=" * 70)

print("""
NOTE:
This experiment is a SAFE SIMULATION.
No real network attack or unauthorized activity
was performed.
""")

```

Step 1: Import Required Libraries

Run the following cell in Google Colab:

```
import random

import pandas as pd

import matplotlib.pyplot as plt

from datetime import datetime, timedelta
```

```
print("Libraries imported successfully.")
```

Step 2: Define IDS Configuration

```
# Trusted IP addresses
```

```
trusted_ips = [

    "10.0.0.10",

    "10.0.0.11",

    "10.0.0.12",

    "10.0.0.13"

]
```

```
# Server being monitored
```

```
server_ip = "10.0.0.50"
```

```
# IDS thresholds
```

```
DOS_THRESHOLD = 50
```

```
PORT_SCAN_THRESHOLD = 5
```

```
print("Trusted IPs:", trusted_ips)
```

```
print("Monitored Server:", server_ip)
```

```
print("DoS Threshold:", DOS_THRESHOLD)
```

```
print("Port Scan Threshold:", PORT_SCAN_THRESHOLD)
```

Step 3: Generate Simulated Network Traffic

The following code creates **normal traffic**, **DoS-like traffic**, and **port-scanning-like traffic**.

```
records = []
```

```
start_time = datetime.now()
```

```
# -----
```

```
# NORMAL TRAFFIC
```

```
# -----
```

```
for i in range(30):
```

```
    source_ip = random.choice(trusted_ips)
```

```
    destination_port = random.choice([80, 443, 22])
```

```
    request_count = random.randint(5, 20)
```

```
    records.append({
        "timestamp": start_time + timedelta(seconds=i),
        "source_ip": source_ip,
        "destination_ip": server_ip,
        "request_count": request_count,
        "unique_ports": random.randint(1, 3),
        "activity_type": "Normal"
    })
```

```
# -----
```

```
# SIMULATED DOS-LIKE TRAFFIC
```

```
# -----
```

```
for i in range(8):
```

```
    source_ip = "10.0.0.99"
```

```
    request_count = random.randint(80, 150)
```

```
    records.append({
```

```
        "timestamp": start_time + timedelta(seconds=40+i),
```

```
        "source_ip": source_ip,
```

```
        "destination_ip": server_ip,
```

```
        "request_count": request_count,
```

```
        "unique_ports": random.randint(1, 2),
```

```
        "activity_type": "DoS-like"
```

```
    })
```

```
# -----
```

```
# SIMULATED PORT SCANNING
```

```
# -----
```

```
for i in range(7):
```

```
    source_ip = "10.0.0.77"
```

```
records.append({  
    "timestamp": start_time + timedelta(seconds=50+i),  
    "source_ip": source_ip,  
    "destination_ip": server_ip,  
    "request_count": random.randint(5, 20),  
    "unique_ports": random.randint(6, 15),  
    "activity_type": "Port Scan-like"  
})
```

```
# -----  
# MIX THE TRAFFIC  
# -----
```

```
random.shuffle(records)
```

```
df = pd.DataFrame(records)
```

```
print("Simulated traffic generated successfully.")
```

```
print("Total Records:", len(df))
```

```
df.head(10)
```

Step 4: Display Simulated Traffic

```
print("SIMULATED NETWORK TRAFFIC")
```

```
print("=" * 70)
```

```
display(df)
```


The table contains:

Field	Description
timestamp	Time of the activity
source_ip	Source of traffic
destination_ip	Target server
request_count	Number of requests
unique_ports	Number of different ports involved
activity_type	Simulated activity category

Step 5: Implement IDS Detection Rules

```
def detect_intrusion(row):
```

```
    alerts = []
```

```
    # Rule 1: Excessive traffic
```

```
    if row["request_count"] > DOS_THRESHOLD:
```

```
        alerts.append("DoS-like Traffic")
```

```
    # Rule 2: Port scanning
```

```

if row["unique_ports"] >= PORT_SCAN_THRESHOLD:
    alerts.append("Port Scanning")

# Rule 3: Untrusted source
if row["source_ip"] not in trusted_ips:
    alerts.append("Untrusted Source")

# Determine severity
if len(alerts) == 0:
    severity = "NORMAL"
    action = "ALLOW"

elif len(alerts) == 1:
    severity = "MEDIUM"
    action = "ALERT"

else:
    severity = "HIGH"
    action = "ALERT + INVESTIGATE"

return ", ".join(alerts) if alerts else "No Suspicious Activity", severity, action

```

Step 6: Apply IDS to Traffic

```

results = df.apply(detect_intrusion, axis=1)

df["detected_activity"] = [r[0] for r in results]
df["severity"] = [r[1] for r in results]
df["recommended_action"] = [r[2] for r in results]

```

```
print("IDS analysis completed.")
```

```
display(df)
```

Step 7: Generate IDS Alerts

```
alerts = df[df["severity"] != "NORMAL"]
```

```
print("INTRUSION DETECTION ALERTS")
```

```
print("=" * 80)
```

```
if len(alerts) == 0:
```

```
    print("No suspicious activities detected.")
```

```
else:
```

```
    for _, row in alerts.iterrows():
```

```
        print(
```

```
            f"[{row['severity']}] " 
```

```
            f"Source: {row['source_ip']} | " 
```

```
            f"Requests: {row['request_count']} | " 
```

```
            f"Ports: {row['unique_ports']} | " 
```

```
            f"Detection: {row['detected_activity']} | " 
```

```
            f"Action: {row['recommended_action']}" 
```

```
        )
```

Step 8: IDS Detection Statistics

```
print("\nIDS DETECTION SUMMARY")
```

```
print("=" * 50)
```

```

print("Total traffic records :", len(df))
print("Normal records      :", len(df[df["severity"] == "NORMAL"]))
print("Medium alerts       :", len(df[df["severity"] == "MEDIUM"]))
print("High alerts         :", len(df[df["severity"] == "HIGH"]))
print("Total alerts        :", len(alerts))

```

Step 9: Analyze Source IPs

```

source_analysis = df.groupby("source_ip").agg(
    total_requests=("request_count", "sum"),
    activity_count=("source_ip", "count"),
    maximum_requests=("request_count", "max")
).reset_index()

source_analysis["status"] = source_analysis["source_ip"].apply(
    lambda ip: "Trusted" if ip in trusted_ips else "Untrusted"
)

display(source_analysis)

```

This helps identify which source generated the highest amount of traffic.

Step 10: Visualize IDS Results

Graph 1: Normal vs Suspicious Traffic

```

traffic_status = df["severity"].apply(
    lambda x: "Normal" if x == "NORMAL" else "Suspicious"
)

status_counts = traffic_status.value_counts()

```

```
plt.figure(figsize=(8, 5))
status_counts.plot(kind="bar")
plt.title("Normal vs Suspicious Network Traffic")
plt.xlabel("Traffic Status")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.show()
```

Graph 2: Activity Type Distribution

```
activity_counts = df["activity_type"].value_counts()
```

```
plt.figure(figsize=(8, 5))
activity_counts.plot(kind="bar")
plt.title("Simulated Network Activity Distribution")
plt.xlabel("Activity Type")
plt.ylabel("Number of Records")
plt.xticks(rotation=0)
plt.show()
```

Graph 3: Severity Distribution

```
severity_counts = df["severity"].value_counts()
```

```
plt.figure(figsize=(8, 5))
severity_counts.plot(kind="bar")
plt.title("IDS Alert Severity Distribution")
plt.xlabel("Severity")
plt.ylabel("Number of Alerts/Records")
plt.xticks(rotation=0)
```

```
plt.show()
```

Step 11: Identify Most Suspicious IP

```
suspicious_ips = df[df["severity"] != "NORMAL"]
```

```
if len(suspicious_ips) > 0:
```

```
    suspicious_ip_count = suspicious_ips["source_ip"].value_counts()
```

```
    print("Most Frequently Detected Suspicious Sources:")
```

```
    print(suspicious_ip_count)
```

```
else:
```

```
    print("No suspicious IP addresses detected.")
```

Step 12: Create IDS Log

A basic IDS should maintain records of detected events.

```
ids_log = df[
```

```
    df["severity"] != "NORMAL"
```

```
][[
```

```
    "timestamp",
```

```
    "source_ip",
```

```
    "destination_ip",
```

```
    "request_count",
```

```
    "unique_ports",
```

```
    "detected_activity",
```

```
    "severity",
```

```
    "recommended_action"
```

```
]]
```

```
print("IDS LOG")
```

```
print("=" * 80)
```

```
display(ids_log)
```

Expected Output

The IDS should classify activities approximately as follows:

Source IP	Request Count	Ports	Expected Detection	Severity
10.0.0.10	10	2	No Suspicious Activity	NORMAL
10.0.0.11	15	1	No Suspicious Activity	NORMAL
10.0.0.99	120	1	DoS-like Traffic, Untrusted Source	HIGH
10.0.0.77	12	10	Port Scanning, Untrusted Source	HIGH
10.0.0.12	18	2	No Suspicious Activity	NORMAL

Because the traffic is randomly generated, the exact values will change each time the program runs.

Practical Tasks / Experiments

Students can perform the following modifications after executing the basic program.

Task 1 – Modify DoS Threshold

Change:

```
DOS_THRESHOLD = 50
```

to:

DOS_THRESHOLD = 30

Observe how the number of alerts changes.

Task 2 – Add a Trusted IP

Add:

"10.0.0.14"

to the trusted IP list and analyze its effect.

Task 3 – Change Port Scan Threshold

Change:

PORT_SCAN_THRESHOLD = 5

to:

PORT_SCAN_THRESHOLD = 3

Compare the number of detected port-scanning activities.

Task 4 – Add a New Detection Rule

Create a rule that detects repeated connections from an untrusted IP.

For example:

IF source IP is untrusted

AND request count > 25

THEN HIGH severity

Task 5 – Compare Detection Results

Compare:

- Normal traffic
- DoS-like traffic
- Port-scanning-like traffic
- Untrusted-source traffic

and identify which activity generated the highest number of alerts.

Observation Table

Sr. No.	Activity	Detection Indicator	IDS Decision	Severity
1	Normal Traffic	Normal request rate	Allow	Normal
2	DoS-like Traffic	High request count	Alert	Medium/High
3	Port Scanning	Multiple ports	Alert	Medium/High
4	Untrusted Source	Unknown IP	Alert	Medium
5	Multiple Indicators	Multiple conditions suspicious	Investigate	High

Result

The practical successfully demonstrates the basic working of an **Intrusion Detection System** using a controlled Python simulation in Google Colab. Synthetic network traffic was generated and analyzed using rule-based detection techniques. The system identified suspicious activities such as **DoS-like excessive traffic, port-scanning-like behavior, and untrusted sources**, generated alerts, classified their severity, maintained an IDS log, and visualized the detection results.

Conclusion

This practical provides a foundational understanding of intrusion detection and demonstrates how security monitoring systems can distinguish normal network behavior from potentially malicious activities. The simulated IDS establishes the basic principles used in real-world security monitoring, while ensuring that all experimentation remains within a safe and controlled Google Colab environment.